Chapter 27 - The Z80 Processor

The Z80 was the dominant rival of the 6502 in the late `1970s` and all through the `1980s`. It powered the ZX Spectrum, the Amstrad CPC, the MSX, the Sega Master System and Game Gear, and a long list of arcade machines. Intuition Engine runs it natively.

It has the same `64`-kilobyte address space as the 6502, the same bank-and-MMIO mapping into the larger Intuition Engine space, and a much richer instruction set: an alternate register bank, two `16`-bit index registers, block-copy and block-search instructions, separate `8`-bit port I/O, and three interrupt modes.

27.1 Architecture

27.1.1 Main register set

Register	Width	Pair	Purpose
`A`	`8`	with `F` as `AF`	Accumulator. Arithmetic and logic.
`F`	`8`	with `A` as `AF`	Flags: `S Z - H - P/V N C`
`B`	`8`	with `C` as `BC`	Counter for block ops; general use
`C`	`8`	with `B` as `BC`	Port number for `IN/OUT (C)`
`D`	`8`	with `E` as `DE`	General purpose
`E`	`8`	with `D` as `DE`	General purpose
`H`	`8`	with `L` as `HL`	Most-used pointer register
`L`	`8`	with `H` as `HL`	Most-used pointer register

27.1.2 Alternate register set

Each register also has a primed shadow: `A'`, `F'`, `B'`, `C'`, `D'`, `E'`, `H'`, `L'`. The shadows are not separately addressable. You exchange them in two atomic moves:

- `EX AF, AF'` swaps `AF` with `AF'`.
- `EXX` simultaneously swaps `BC`/`DE`/`HL` with `BC'`/`DE'`/`HL'`.

The conventional use is for fast interrupt response: the handler swaps banks on entry, does its work in the alternate set, and swaps back on exit, never touching the foreground set's contents.

27.1.3 Index, special, and control

Register	Width	Purpose
`IX`	`16`	Index register (use with displaced addressing)
`IY`	`16`	Index register (use with displaced addressing)
`SP`	`16`	Stack pointer
`PC`	`16`	Program counter
`I`	`8`	Interrupt vector page register (used in `IM 2`)

Register	Width	Purpose
R	8	Refresh counter - increments on every M1 cycle

27.1.4 Flag register

Bit	Name	Meaning
7	S	Sign - copy of bit 7 of result
6	Z	Zero - set if result was zero
5	-	Undefined (copy of bit 5 of result on most ops)
4	H	Half carry - carry from bit 3 to bit 4
3	-	Undefined (copy of bit 3 of result on most ops)
2	P/V	Parity (logic ops) / overflow (arithmetic)
1	N	Subtract - set after subtraction; used by `DAA`
0	C	Carry

27.2 The Intuition Engine memory map for the Z80

The Z80 sees the same 64-K layout as the 6502:

Range	Purpose
`0x0000–0x1FFF`	Free RAM
`0x2000–0x3FFF`	Bank 1 window (8 KiB)
`0x4000–0x5FFF`	Bank 2 window (8 KiB)
`0x6000–0x7FFF`	Bank 3 window (8 KiB)
`0x8000–0xBFFF`	VRAM bank window (16 KiB)
`0xC000–0xEFFF`	Free RAM
`0xF000-0xFFF9`	MMIO window (maps to `0xF0000-0xF0FF9`)

As on the 6502, the CPU adapter intercepts the bank-control bytes inside the MMIO window before bus translation: `BANK1_REG_LO` `0xF700`, `_HI` `0xF701`; `BANK2_REG` `0xF702`/`0xF703`; `BANK3_REG` `0xF704`/`0xF705`; `VRAM_BANK_REG` `0xF7F0`. Writes to those bytes change the matching bank window. All other addresses in `0xF000-0xFFF9` reach the corresponding MMIO register on the bus.

There is no Z80 memory-mapped path to `TERM_OUT` (`0xF0700`); the `$F700` byte is captured by the bank intercept. Bare Z80 code that needs to display text uses the ULA or VGA chips, the SoundChip beeper, or one of the I/O ports listed below.

The Z80 also has the architectural 256-port I/O space. Intuition Engine maps useful devices into ports as well as into memory:

Ports	Device
`0xA0–0xAA`	VGA registers
`0xFE`/`0xFD`/`0xBE`/`0xFA–0xFC`	ULA control and VRAM data port

The PSG, SID, TED and ANTIC chips also expose port-I/O selectors; each engine's chapter has the specifics.

27.3 Addressing modes

Z80 addressing modes:

Mode	Example	Meaning
Immediate	`LD A, n`	Literal 8-bit
Immediate extended	`LD HL, nn`	Literal 16-bit
Modified zero page	`RST 38h`	Call to a fixed 8-byte slot in page 0
Relative	`JR e`	Signed 8-bit PC offset (used by JR/DJNZ)
Extended	`LD A, (nn)`	16-bit absolute address
Indexed	`LD A, (IX+d)`	`IX`/`IY` plus signed 8-bit displacement
Register	`LD A, B`	Register direct
Register indirect	`LD A, (HL)`	Byte at the address in a register pair
Implied	`CCF`	No operand
Bit	`BIT b, r`	Single bit of a register or `(HL)`

The two index registers `IX` and `IY` give the Z80 a true displaced addressing mode that the 6502 lacks.

27.4 Instruction groups

The Z80 has about 158 distinct mnemonics with 694 opcode forms. Appendix G has the full opcode table; this section names the groups and what each does.

27.4.1 8-bit load group

`LD r, r'` (any 8-bit register to any 8-bit register); `LD r, n` (immediate); `LD r, (HL)`; `LD (HL), r`; `LD A, (BC)`; `LD A, (DE)`; `LD A, (nn)`; `LD (BC), A`; `LD (DE), A`; `LD (nn), A`; `LD A, I`; `LD A, R`; `LD I, A`; `LD R, A`.

27.4.2 16-bit load group

`LD rp, nn` (pair immediate); `LD HL, (nn)`; `LD rp, (nn)`; `LD (nn), HL`; `LD (nn), rp`; `LD SP, HL`; `PUSH rp`; `POP rp`.

27.4.3 Exchange, block transfer, block search

Mnemonic	Operation
`EX DE, HL`	Swap `DE` and `HL`
`EX AF, AF'`	Swap `AF` with `AF'`
`EXX`	Swap `BC`/`DE`/`HL` with the shadow set
`EX (SP), HL`	Swap `HL` with the top word of the stack
`LDI`/`LDIR`/`LDD`/`LDDR`	Block move (HL to DE, BC bytes)
`CPI`/`CPIR`/`CPD`/`CPDR`	Block compare (search for A)

`LDIR` and `CPIR` repeat until `BC` reaches zero. `LDDR` and `CPDR` are their decrementing counterparts.

27.4.4 8-bit arithmetic and logic

`ADD A, r`/`ADC A, r`/`SUB r`/`SBC A, r`/`AND r`/`OR r`/`XOR r`/ `CP r`/`INC r`/`DEC r`. Each is available with any of the addressing modes that produce an 8-bit operand.

`DAA` adjusts the accumulator after a BCD operation.

27.4.5 16-bit arithmetic

`ADD HL, rp`; `ADC HL, rp`; `SBC HL, rp`; `ADD IX, rp`; `ADD IY, rp`; `INC rp`; `DEC rp`.

27.4.6 Rotate and shift

`RLCA`, `RLA`, `RRCA`, `RRA` operate on `A`. The `CB`-prefix group `RLC r`, `RL r`, `RRC r`, `RR r`, `SLA r`, `SRA r`, `SRL r`, `SLL r` (`SLL` is undocumented but supported) operates on any `8`-bit register or `(HL)`/`(IX+d)`/`(IY+d)`.

27.4.7 Bit set, reset, and test

`BIT b, r` (set `Z` from bit `b`), `SET b, r`, `RES b, r`. Bit number `b` is `0`–`7`; register `r` is any `8`-bit register or `(HL)`/`(IX+d)`/`(IY+d)`.

27.4.8 Jump, call, return, restart

Mnemonic	Operation
`JP nn`	Unconditional absolute jump
`JP cc, nn`	Conditional absolute jump
`JP (HL)`/`JP (IX)`/`JP (IY)`	Register-indirect jump
`JR e`	Unconditional relative jump (`-126` to `+129`)
`JR cc, e`	Conditional relative jump (`cc` in `NZ`/`Z`/`NC`/`C`)
`DJNZ e`	Decrement `B`, branch if non-zero
`CALL nn`/`CALL cc, nn`	Call (push PC, jump)
`RET`/`RET cc`	Return
`RETI`	Return from interrupt (signals the device)
`RETN`	Return from non-maskable interrupt
`RST p`	Call to a fixed page-`0` slot (`p` in `00h`/`08h`/`10h`/`18h`/`20h`/`28h`/`30h`/`38h`)

The condition codes for `JP cc`, `JR cc`, `CALL cc`, `RET cc`: `NZ`, `Z`, `NC`, `C`, `PO`, `PE`, `P`, `M`.

27.4.9 Port I/O

Mnemonic	Operation
`IN A, (n)`	Read port `n`, address bits `8`–`15` = `A`
`OUT (n), A`	Write `A` to port `n`
`IN r, (C)`	Read port `BC`, set flags from result

Mnemonic	Operation
`OUT (C), r`	Write `r` to port `BC`
`INI`/`INIR`/`IND`/`INDR`	Block input (port to memory)
`OUTI`/`OTIR`/`OUTD`/`OTDR`	Block output (memory to port)

27.4.10 CPU control

Mnemonic	Operation
`NOP`/`HALT`	No op / wait for interrupt
`DI`/`EI`	Disable / enable interrupts
`IM 0`/`IM 1`/`IM 2`	Set interrupt mode
`SCF`/`CCF`	Set / complement carry
`CPL`	Ones-complement of `A`
`NEG`	Twos-complement of `A`

27.5 Interrupts

The Z80 has three interrupt modes:

- **`IM 0`** - the interrupting device puts an 8-bit opcode on the data bus. The CPU executes it. The convention is a `RST n`, giving a fixed 8-slot vector.
- **`IM 1`** - every IRQ is handled as `RST 38h`. The vector is the byte at `0x0038`. This is the mode used by the ZX Spectrum.
- **`IM 2`** - the device supplies a vector byte. The CPU forms the address `(I << 8) | vector` and reads a 16-bit handler pointer from there. This gives 128 independent vectors.

`NMI` always jumps to `0x0066`, ignoring the interrupt mode and the global enable flag.

Two enable flip-flops `IFF1` and `IFF2` track maskable-interrupt state across `NMI` handlers: `NMI` copies `IFF1` to `IFF2` and clears `IFF1`; `RETN` copies `IFF2` back into `IFF1`. The ordinary handler uses `EI` and `RETI`.

27.6 Undocumented opcodes

The Z80 silicon decodes a number of opcodes that the original Zilog manual does not list. Intuition Engine implements the common ones, including:

- The undocumented `SLL r` (shift left logical, alias `SL1`).
- `IXH`, `IXL`, `IYH`, `IYL` as separate 8-bit halves of the index registers, accepted as operands by ordinary `LD`, `ADD`, `SUB`, etc.
- The `ED`-prefix block `IN F, (C)` and `OUT (C), 0`.

Appendix G has the complete listing.

27.7 A small example

A Z80 program that beeps the SN76489 once and then spins:

```
SN_DATA   equ 0xE4              ; SN76489 data port
SN_STATUS equu 0xE5              ; SN76489 status port (read-only)

        .org    0x1000

start:  ld      a, 0x9F           ; channel 1 attenuation: 0 dB
        out     (SN_DATA), a
        ld      a, 0x80 | 0x0E    ; channel 0 tone latch, low 4 bits
        out     (SN_DATA), a
        ld      a, 0x01           ; tone high 6 bits
        out     (SN_DATA), a

halt:   jp      halt
```

The Z80 stops by running an infinite loop. The image executor at `0x1000` provides the startup stub for `.ie80` images.

27.8 What comes next

Chapter 28 covers the M68K, the `16`/`32`-bit processor that replaced both the 6502 and the Z80 in the late `1980s` machines: the Atari ST, the Amiga, the early Apple Macintosh. The M68K has `32`-bit data and address registers and a flat `16`-megabyte (or, on the 68020, full `32`-bit) address space, which means it sees Intuition Engine's MMIO map directly and needs no banking machinery.